

Stepping Stone Graph: A Graph for Finding Movement Corridors using Sparse Trajectories

SAMEERA KANNANGARA, EGEMEN TANIN, AARON HARWOOD, and SHANIKA KARUNASEKERA, The University of Melbourne

There are many real world applications that require identifying public movements such as identifying movement corridors in cities and most popular paths. If one is not given user trajectories but rather sporadic location data, such as location-based social network data, finding movement related information becomes difficult. Rather than processing all points in a dataset given a query, a clever approach is to construct a graph, based on user locations, and query this graph to answer questions such as shortest paths, most popular paths, and movement corridors. Shortest path graph is one of the popular graphs. However, the shortest path graph can be inefficient and ineffective for analysing movement data, as it calculates the graph edges considering the shortest paths over all the points in a dataset. Therefore, edge sets resulting from shortest path graphs are usually very restrictive and not suitable for movement analysis because of its global view of the dataset. We propose the stepping stone graph, which calculates the graph considering point pairs rather than all points; the stepping stone graph focuses on possible local movements, making it both efficient and effective for location-based social network related data. We demonstrate its capabilities by applying it in the Location-Based Social Network domain and comparing with the shortest path graph. We also compare its properties to a range of other graphs and demonstrate how stepping stone graph relates to Gabriel graph, relative neighbourhood graph, and Delaunay triangulation.

CCS Concepts: • **Information systems** → **Geographic information systems**;

Additional Key Words and Phrases: Moving objects, shortest path, graphs

ACM Reference format:

Sameera Kannangara, Egemen Tanin, Aaron Harwood, and Shanika Karunasekera. 2019. Stepping Stone Graph: A Graph for Finding Movement Corridors using Sparse Trajectories. *ACM Trans. Spatial Algorithms Syst.* 5, 4, Article 23 (December 2019), 24 pages.

<https://doi.org/10.1145/3324883>

1 INTRODUCTION

Location-Based Social Network (LBSN) data provides an inexpensive and rich data source as LBSN typically has voluntarily generated content inspired by the users' surroundings along with the locality where the content is generated. The rise of LBSN highlights new GIScience considerations when catering to a tighter integration of LBSN and geography [20]. LBSN data provides a valuable

This research is funded in part by the Defence Science and Technology Group, Edinburgh, South Australia, under contract MyIP:6104.

Authors' address: S. Kannangara, E. Tanin, A. Harwood, and S. Karunasekera, School of Computing and Information Systems, University of Melbourne, Victoria 3010, Australia; emails: kannangarad@student.unimelb.edu.au, {etanin, aharwood, karus}@unimelb.edu.au.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Association for Computing Machinery.

2374-0353/2019/12-ART23 \$15.00

<https://doi.org/10.1145/3324883>

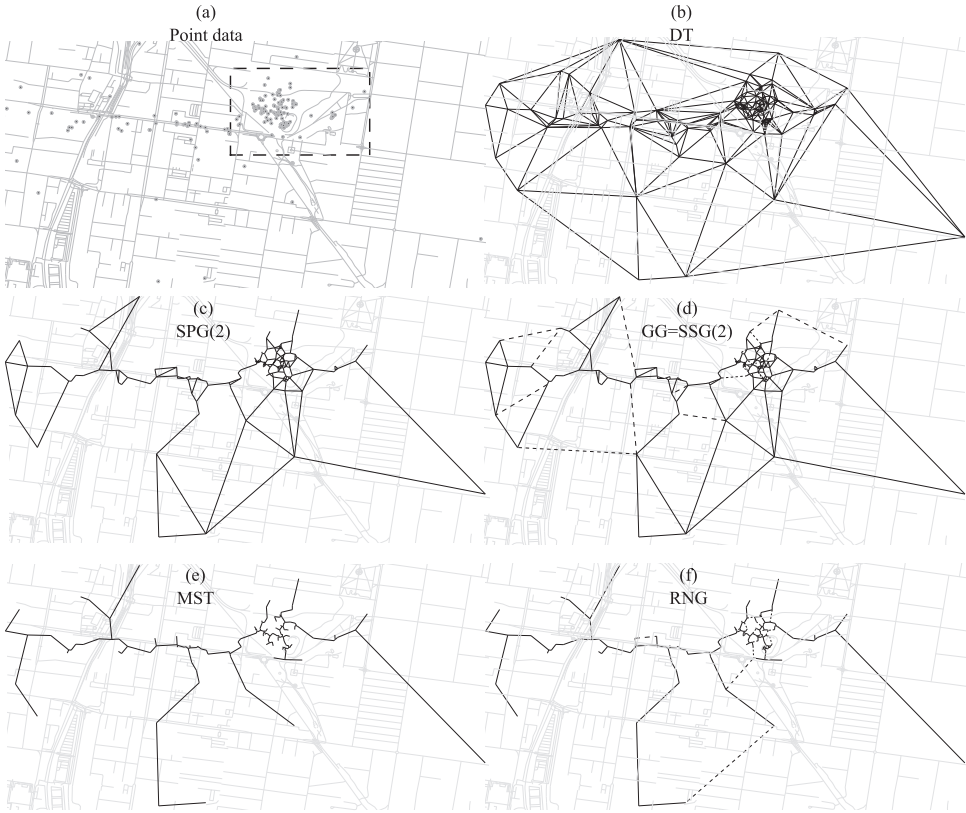


Fig. 1. A point set (a), with a rectangle in dashed lines indicating the location where a music festival was held, and a set of connected neighbourhood skeletons (b)–(f). Dashed lines in (d) and (f) indicate additional edges inferred by our proposed Stepping Stone Graph compared to the Shortest Path Graph, (c) and (e).

source for establishing situational awareness during interesting events [14]. Since different users have different posting patterns, proposing methods to analyse public movement using LBSN data is difficult [4]. A common problem is that most users do not post content frequently enough to deduce any trajectory information. We propose to combine location information from different users to establish a movement network, as there is an increased interest in mining movement from spatial data [7].

Often spatial data is provided as a distinct point set in the LBSN domain. Graphs, in which the relationships between points can be represented as edges, are useful for representing and processing such point data. A *neighbourhood graph* is one where the edges are inferred based on the relationships defined on the point set provided for analysis. Distance-based connected neighbourhood graphs are useful for inferring and processing movement networks between a set of distinct points [5]. Delaunay Triangulation (DT), Minimum Spanning Tree (MST), Gabriel Graph (GG), and Relative Neighbourhood Graph (RNG) are existing well-known distance-based connected neighbourhood graphs which are widely used to infer and process a movement network between point data [5]. These graph approaches are *static* as they produce a unique movement network, represented as a *skeleton*, for any given point set. As an example, Figure 1(a) shows a location set collected from Twitter relating to a music festival. Figure 1(b), (d), (e), and (f) represent DT, GG, MST, and RNG skeletons, respectively. Each neighbourhood skeleton is obtained by connecting

the endpoints of the inferred edges using straight line segments which are referred to as the geometric realization of a neighbourhood graph and shows the geometric “shape” of the point set.

As opposed to the static graph approaches above, variable graph approaches produce a spectrum of possible skeletons, usually based on a single parameter. Hence, variable graphs are more versatile [11]. The Shortest Path Graph ($SPG(t)$) [6] is such a variable graph, proposed with the idea of inferring edges between a point set such that the shortest path taken over the inferred edges will roughly align with the shortest path taken over the imprecise region represented by the point set. Characteristics of the inferred edge set vary based on a single user provided parameter $t \geq 1$. $SPG(t)$ converges to a MST of the given point set as $t \rightarrow \infty$. We have found that the structure imposed by $SPG(t)$, when inferring edges using its distance-based global criteria, is useful for some use cases, such as delineating imprecise regions [6] and overlapping set visualization [15], but is over-burdensome or unnecessary in other use cases such as movement analysis. Figure 1(c) and (e) represent skeletons generated using $SPG(t)$ with $t = 2, \infty$, respectively.

To address the deficiency of existing approaches when considering the movement corridor use case, in this article we propose a new type of variable graph, which we refer to as the Stepping Stone Graph ($SSG(d)$) with parameter $d \geq 1$. Similar to the fact that $SPG(t)$ converges to MST , our $SSG(d)$ converges to RNG as $d \rightarrow \infty$, thus generating the RNG . Note that RNG provides a structure more similar to the human perception of a point set than the MST [23], which is advantageous. Therefore, the spectrum of skeletons inferred by our $SSG(d)$ is likely to be more perceptually relevant compared to $SPG(t)$. We also show in our experiments that $SSG(d)$ is easier and faster to build and gives better results in processing movement analysis related queries. This is because as the name suggests $SSG(d)$ focusses on where people have been and can step on for movement rather than any global characteristic.

To achieve this, $SSG(d)$ infers edges between a point set using a distance-based local criterion based on the inherent properties of movement networks. $SPG(t)$, on the other hand, creates connections without enough consideration whether these connections can lead to a realistic step between two “stepping stones” within an area. Thus, we assume that social network data gives us partial data per individual user in terms of its path but with a good picture of where people could be in an event in a city. Intuitively, for all endpoint pairs, the value of d indicates the preference of inferring a longer alternative path with less distance between all point pairs on the path compared to the direct distance between the endpoint pair. This is useful when we have a very dense point set to cull some connections. Similarly to $SPG(t)$, as d increases, the number of edges in $SSG(d)$ monotonically decreases and therefore the path length between any two non-adjacent points in the skeleton monotonically increases. Figure 1(c) and (d) and Figure 1(e) and (f) depict $SPG(t)$ and $SSG(d)$ with $t = d = 2, \infty$, respectively. For each $d = t$ pair, additional edges inferred by $SSG(d)$ which are not inferred by $SPG(t)$ are highlighted using dashed lines, shown in Figure 1(d) and (f). It is important to note that GG is a special case of $SSG(d)$ when $d = 2$.

We use publicly available LBSN data to evaluate the utility of $SPG(t)$ and $SSG(d)$ for inferring movement networks. We show that $SSG(d)$ performs better at applications such as popular path identification. Also given a DT , $SSG(d)$ can infer a planar edge embedding in $O(n)$ time where n is the number of points, whereas $SPG(t)$ takes $O(n^2 \log n)$. From our experiments, it is evident that $SSG(d)$ has a lower and more stable spanning ratio [2] compared to $SPG(t)$. Thus, $SSG(d)$ is more effective for inferring and processing the movement network compared to $SPG(t)$. We performed experiments on the analysis of movement networks for locations influenced by an event, and movement networks between localities posted within a city. In both experiments, $SSG(d)$, for the case of $d = t$, inferred paths are more similar to the real world paths, compared to $SPG(t)$. Our graph is also faster to compute. It also generalizes the RNG .

This article is an extended version of the work presented in [10]. This article contains proofs for all the theorems presented and one new theorem. In the new theorem, non-planar regions of $SSG(d)$ and the lune-based β -skeleton are compared and the relationship between parameters d and β where $SSG(d)$ become a subgraph of lune-based β -skeleton is shown. With these proofs and the new theorem, readers will gain a deeper understanding of the properties of the $SSG(d)$. Furthermore, this article introduces five new queries on the Stepping Stone Graph. Two of these queries are extended concepts of nearest neighbour queries. Namely, *reverse nearest neighbour queries* and *group nearest neighbour queries*. The results of these two queries exhibit the suitability of $SSG(d)$ in the standard extensions of nearest neighbour queries. The third query we introduce is *inferring road networks*, which is an extension of refined movement corridor finding. This operation can be used to infer road networks using location data posted on the road network within an area where we do not have road network information. The other two queries relate to the processing trajectory traces for gaining combined knowledge. The first query of this category is *trajectory clustering*, where given a set of trajectories we group similar trajectory traces together and come up with a representative trajectory for each group. The next query is *group movement detection*, which is an extension of the technique used for trajectory clustering. In this application, we incorporate the temporal aspect of the trajectories to find trajectories both spatially and temporally close and detect users who have moved together. These two queries exhibit usability of $SSG(d)$ in the trajectory similarity processing domain. Using real world datasets and synthetic data, we show that $SSG(d)$ outperforms $SPG(t)$ in all five query types.

2 RELATED WORK

2.1 Spatial Analysis of LBSN Data

Our work focuses on visualizing the possible movement network using variable neighbourhood graphs. Following is some research that uses LBSN data to analyse interesting events to which the proposed skeleton can be added to visualise potential user flow patterns. MacEachren et al. [14] present results of a survey conducted on potential uses of social media in crisis management, with the participation of crisis management professionals. In order to cater to the requirements gathered from the survey, the authors propose a prototype system named SensePlace2. SensePlace2 allows the analyst to query the social media stream and visualise retrieved social media posts, a tag cloud, and post distribution over a map. Chae et al. [4] present another visual analytic system for public behaviour analysis using social media data. This system features a map which allows analysts to add indicators to identify points of interest in the analysed areas and movements of natural disasters. Initially, they have tried to reveal public movement flows during interesting events. But, they were unable to find meaningful flows due to the movement randomness and the visual clutter of the flows. In both the above systems, to provide a summarized view of post distribution, authors have noted the usefulness of geospatial heatmaps. Moreover, the colouring overlay of the heatmap is computed using statistical methods.

2.2 Neighbourhood Graphs

Neighbourhood Graphs infer edges between points based on some criteria. In neighbourhood graphs, edge inferring criteria can be defined per point, per point pair, or per all points in the sample. Neighbourhood graphs that infer edges based on the absence of other vertices inside a region surrounding the endpoints of the edge are referred as Empty Region Graphs (ERG) [3]. When using an empty region as the evaluation criteria for edge inferring, it is important to note whether the boundary of the evaluation region is considered to be empty or not. If the boundary is considered to be empty, then the evaluation region is referred to as a closed region and as an open

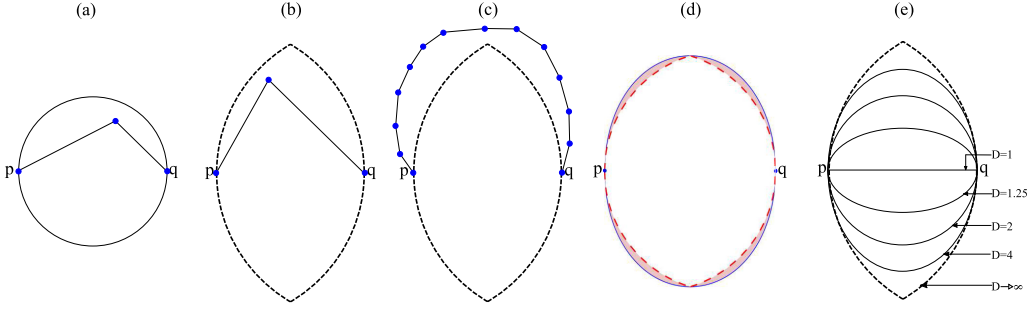


Fig. 2. p and q are two points in space. Examples of (a) GG , (b) RNG , and (c) $SPG(2)$ of point sets with (a) closed circle, (b) and (c) open lune neighbourhoods, (d) β neighbourhood (at $\beta = 1.41421$) with dashed line on top of $DN(4)$, and (e) graphical representation of $DN(pq,d)$.

region otherwise. It is a common practice to represent the boundaries of open and closed regions by dashed and solid lines, respectively, when graphically representing them.

Both GG [8] and RNG [23] are static ERG that are useful for analysing the shape of a point set. GG is originally proposed as a tool for geographic variation analysis. The authors emphasise that the connectedness and planarity (non-existence of overlapping edges in skeleton) are crucial to make inferred edges intuitive for geographic data. As the empty region for GG , authors use the simple definition closed circle where the inferring edge between a point pair is a diameter of the circle. RNG , which is introduced as a mechanism that can infer a structure close to human perception of a point set, uses open lune as the empty neighbourhood. Closed circle and open lune neighbourhoods for a vertex pair p, q , are shown in Figure 2(a) and (b), respectively. In [3], closed circle and open lune are proved as the tight regions for planar and connected edge inferring, respectively. Note that $RNG \subseteq GG$. As later shown, $SSG(2) \equiv GG$ and $SSG(\infty) \equiv RNG$.

β -Skeletons [11] are proposed to analyse the shape of a point set and presented as a framework for the neighbourhood graphs which infer edges based on the circle-based empty regions. As opposed to the distance-based definition of $SSG(d)$, β -Skeletons have a geometry-based definition. Thus, $SSG(d)$ is more robust compared to β -Skeletons as later shown when analysing properties of $SSG(d)$. The authors present two categories of neighbourhoods. The lune-based neighbourhoods are defined as an intersection of two circles, and the circle-based neighbourhoods are defined as a union of two circles. With the lune-based definition, GG is $\beta = 1$ closed neighbourhood skeleton and RNG is $\beta = 2$ open neighbourhood skeleton.

$SPG(t)$ is first proposed by Mark de Berg et al. as the base skeleton of a delineating mechanism to identify the boundary and cavities within an imprecise region, given as a positive locality sample [6]. When the weight of an edge is defined as its Euclidean length raised to the power of the parameter $t \geq 1$, the edge is present in $SPG(t)$ if and only if it is the least weight path between its endpoints. The authors show using empirical results that $SPG(2)$ is better for delineating an imprecise region, compared to both Kernel Density Estimation (KDE) and GG . $SPG(2)$ performs better than GG and β -Skeletons for delineating imprecise regions, because of its global criteria. Presenting the $SPG(t)$ -based delineating mechanism, the authors highlight that the connectedness of the resulting structure, created for analysis, is a desirable property when processing an imprecise region provided as a positive locality sample [6]. Since open lune is the upper bound of the empty regions to ensure a connected skeleton, $SPG(2)$ should be compared against RNG . In Figure 2(c), $SPG(2)$ does not contain an edge between p and q . However, for the same situation RNG infers an edge. Therefore, we can say that the RNG provides more alternative paths

compared to $SPG(2)$ when inferring the movement network among the point set in Figure 2(c). In the discussion, the authors note that the quality of the result generated using the proposed delineating method depends heavily on the parameter selection of outlier filtering. Later, $SPG(t)$ skeletons with varying t are used as a base structure for overlapping set visualization [15].

Like $SPG(t)$ that converges to MST as $t \rightarrow \infty$, $SSG(d)$ converges to RNG as $d \rightarrow \infty$. $SSG(d)$ infers a graph closer to human perception of a point set compared to $SPG(t)$ [23]. A key disadvantage of $SPG(t)$ is that using a global criterion in its inference algorithm results in undesirable running times such as $O(n^2 \log n)$ to infer planar edges. Thus, generating $SPG(t)$ with varying t values is a computationally intensive task. In contrast, our proposed $SSG(d)$ utilizes a local evaluation criterion, which is inferred in $O(n)$ time given the DT for planar edge embedding.

3 STEPPING STONE GRAPH

We consider all location points provided for the analysis as stepping stones that need to be linked in a traversable manner. So we focus on connecting these stepping stones such that a travelling entity can find a path between two locations.

3.1 Definitions

We construct $SSG(d)$ in the form of an undirected graph $G(V, E)$ where $V \subseteq \mathbb{R}^2$ represents a given point set and E represents inferred edges between the points. An edge between two endpoints $p, q \in V$ is represented as $pq \in E$. Length l_{pq} represents the Euclidean distance between two points.

First, we define the *diversion neighbourhood* in Euclidean space, which is the main construct necessary for defining $SSG(d)$, an area between two points which varies based on a single parameter $d \geq 1$.

Definition 3.1 (Diversion Neighbourhood). For $p, q \in \mathbb{R}^2$, the diversion neighbourhood of pq at $d \in \mathbb{R} : d \geq 1$, denoted $DN(pq, d)$ or simply $DN(d)$, is defined as the region

$$DN(pq, d) = \left\{ z \in \mathbb{R}^2 : l_{pz}^d + l_{zq}^d \leq l_{pq}^d \right\}. \quad (1)$$

A graphical representation of $DN(pq, d)$ for varying d is shown in Figure 2(e). As the figure shows, $DN(d)$ enlarges from a straight line connecting the endpoints to the open lune neighbourhood between the endpoints. Note that we use $\leq$ instead of $<$ in Equation (1), making $DN(d)$ a closed region. Next, we define the Stepping Stone Graph using $DN(d)$.

Definition 3.2 (Stepping Stone Graph). For $V \subseteq \mathbb{R}^2$, the Stepping Stone Graph of V at $d \in \mathbb{R} : d \geq 1$, denoted $SSG(V, d)$ or simply $SSG(d)$, is defined as an undirected graph with points V , such that for each point pair $p, q \in V$:

$$pq \text{ is an edge of } SSG(V, d) \text{ iff } DN(pq, d) \cap V \setminus \{p, q\} = \emptyset.$$

Intuitively, if $DN(d)$ for a given d and endpoint pair contains no other points, then $SSG(d)$ will have an edge between that endpoint pair.

Since $DN(d)$ becomes an open lune as $d \rightarrow \infty$, $SSG(d)$ is a connected graph, due to Theorem 25 in [3]. After creating $SSG(d)$, edges longer than a given threshold can be filtered out to accommodate travel distance requirements of a travelling entity [5]. The diversion neighbourhood is effectively a local criterion when selecting edges in $SSG(d)$, as opposed to the global criterion in $SPG(t)$, and therefore in some cases $SSG(d)$ provides more alternative paths between a point set compared to $SPG(t)$, and $SSG(d)$ never provides less alternative paths than $SPG(t)$.

By definition of $DN(d)$, as d increases, the travel distances over the shortest paths taken from $SSG(d)$ between all endpoint pairs monotonically increase. At the same time, the number of points on the shortest paths monotonically increases. Therefore, as d increases, the average distance to be

travelled between a point pair over the shortest path taken from $SSG(d)$ monotonically decreases. Hence, $SSG(d)$ furnishes alternative paths where, for a given endpoint pair, each of the path edges in the alternative path is always shorter than the direct path between the endpoint pair.

3.2 Stepping Stone Graph Properties

As d increases, the area covered by $DN(pq, d)$ increases symmetrically around both the perpendicular bisector and the direct connecting line of pq . This leads to $SSG(d')$ being more restrictive than $SSG(d)$ for $d' > d$. Therefore, as d increases, by definition the number of edges in $SSG(d)$ monotonically decreases. Hence, the following theorem results.

THEOREM 3.3. *For $1 \leq d \leq d'$, $SSG(d') \subseteq SSG(d)$.*

PROOF. Define the *edge weight* of pq with respect to d' as $l_{pq}^{d'}$, for some $d' \geq 1$. Assume that for all $z \in V \setminus \{p, q\}$, $l_{pz}^{d'} + l_{zq}^{d'} > l_{pq}^{d'}$. In this case, pq is an edge in $SSG(d')$. Now we show that for $d \leq d'$, pq is also an edge in $SSG(d)$. Let us write $d = d' \epsilon$ where $\frac{1}{d'} \leq \epsilon \leq 1$. Then we need to show that

$$\begin{aligned} l_{pz}^{d' \epsilon} + l_{zq}^{d' \epsilon} &> l_{pq}^{d' \epsilon}, \\ \frac{l_{pz}^{d' \epsilon} + l_{zq}^{d' \epsilon}}{l_{pq}^{d' \epsilon}} &> 1, \\ \left(\frac{l_{pz}}{l_{pq}}\right)^{d' \epsilon} + \left(\frac{l_{zq}}{l_{pq}}\right)^{d' \epsilon} &> 1, \\ \left(\left(\frac{l_{pz}}{l_{pq}}\right)^{d' \epsilon} + \left(\frac{l_{zq}}{l_{pq}}\right)^{d' \epsilon}\right)^{\frac{1}{\epsilon}} &> 1. \end{aligned} \tag{2}$$

Since the function $x \mapsto x^\beta$ is subadditive for $\beta \geq 1$, then

$$\left(\left(\frac{l_{pz}}{l_{pq}}\right)^{d' \epsilon} + \left(\frac{l_{zq}}{l_{pq}}\right)^{d' \epsilon}\right)^{\frac{1}{\epsilon}} \geq \left(\frac{l_{pz}}{l_{pq}}\right)^{d'} + \left(\frac{l_{zq}}{l_{pq}}\right)^{d'}.$$

We know the right-hand side is greater than 1 due to our initial assumption and therefore Equation (2) is true. Therefore, pq is also an edge in $SSG(d)$ and this completes the proof. $\square$

By definition of $DN(d)$ and by the Thales' theorem, $DN(pq, 2)$ becomes a closed circumcircle of pq and as $d \rightarrow \infty$, $DN(d)$ becomes the open lune. Therefore, the following two theorems result.

THEOREM 3.4. $SSG(2) \equiv GG$.

THEOREM 3.5. As $d \rightarrow \infty$, $SSG(d) \rightarrow RNG$.

By Theorem 3.5, we refer to $\lim_{d \rightarrow \infty} SSG(d)$ as $SSG(\infty) \equiv RNG$ in later sections. The following lemmas are trivial due to Theorems 24 and 25 of the Empty Region Graph study [3].

LEMMA 3.6. *For $d \geq 2$, $SSG(d)$ is planar.*

LEMMA 3.7. *For all $d \geq 1$, $SSG(d)$ is connected.*

By combining Lemmas 3.6 and 3.7, the range of d where $SSG(d)$ is both connected and planar can be deduced.

LEMMA 3.8. *For $d \geq 2$, $SSG(d)$ is both planar and connected.*

Now we compare SSG to SPG .

THEOREM 3.9. For $d \leq t$, $SPG(t) \subseteq SSG(d)$.

PROOF. Define the *edge weight* of pq with respect to t as l_{pq}^t , for some $t \geq 1$. Assume that for all $z \in V \setminus \{p, q\}$, $l_{pz}^t + l_{zq}^t > l_{pq}^t$. In this case, pq is an edge in $SPG(t)$. Now we show that for $d \leq t$, pq is also an edge in $SSG(d)$. Let us write $d = t\epsilon$ where $\frac{1}{t} \leq \epsilon \leq 1$. Then we need to show that

$$\begin{aligned} l_{pz}^{t\epsilon} + l_{zq}^{t\epsilon} &> l_{pq}^{t\epsilon}, \\ \frac{l_{pz}^{t\epsilon} + l_{zq}^{t\epsilon}}{l_{pq}^{t\epsilon}} &> 1, \\ \left(\frac{l_{pz}}{l_{pq}}\right)^{t\epsilon} + \left(\frac{l_{zq}}{l_{pq}}\right)^{t\epsilon} &> 1, \\ \left(\left(\frac{l_{pz}}{l_{pq}}\right)^{t\epsilon} + \left(\frac{l_{zq}}{l_{pq}}\right)^{t\epsilon}\right)^{\frac{1}{\epsilon}} &> 1. \end{aligned} \tag{3}$$

Since the function $x \mapsto x^\beta$ is subadditive for $\beta \geq 1$, then

$$\left(\left(\frac{l_{pz}}{l_{pq}}\right)^{t\epsilon} + \left(\frac{l_{zq}}{l_{pq}}\right)^{t\epsilon}\right)^{\frac{1}{\epsilon}} \geq \left(\frac{l_{pz}}{l_{pq}}\right)^t + \left(\frac{l_{zq}}{l_{pq}}\right)^t.$$

We know the right-hand side is greater than 1 due to our initial assumption and therefore Equation (3) is true. Therefore, pq is also an edge in $SSG(d)$ and this completes the proof. $\square$

Now we consider the relationship between $SSG(d)$ and β -Skeletons. The empty regions used for both the lune-based β -Skeleton [11] and $SSG(d)$ are symmetric around the perpendicular bisector of the straight line connecting point pair. Using this similarity, we analysed how these neighbourhoods relate to each other for $d \geq 2$. As depicted in Figure 2(d), we observed that the β -Skeleton touches $DN(d)$ only on four points in the range of $\beta = (1, 2)$. Therefore, we can say that $DN(d)$ is more robust compared to the lune-based neighbourhood of β -Skeleton in the range $\beta = (1, 2)$. We used the fact that Euclidean distances between these four points should be equal, to analyse the relationship between β and d . As a result of this analysis, we arrived at the following theorem.

THEOREM 3.10. $\forall d \geq 2 \wedge \beta \leq 2^{(1-2/d)},$

$$SSG(d) \subseteq \text{lune-based } \beta\text{-skeleton}.$$

PROOF. Figure 3 displays the schematic used to calculate Theorem 3.10. A Cartesian coordinate system is used with the inferring edge of length $2L$ as the X axis. The midpoint of the inferring edge is taken as the origin. β neighbourhood is indicated by the dashed line plot and $DN(d)$ is indicated by the solid line plot.

The point where β neighbourhood touches $DN(d)$ above the edge is taken as $(0, Y)$. As boundaries of both neighbourhoods touch each other, Y calculated using both β neighbourhood (Y_β) and $DN(d)$ (Y_d) should be equivalent.

$$\begin{aligned} Y_\beta &= Y_d, \\ L\sqrt{2\beta - 1} &= L\sqrt{4^{(1-1/d)} - 1}, \\ \beta &= 2^{(1-2/d)}, \\ \therefore \forall 2 \leq d \wedge \beta &\leq 2^{(1-2/d)}, \end{aligned}$$

$$SSG(d) \subseteq \text{lune based } \beta\text{-skeleton}.$$

$\square$

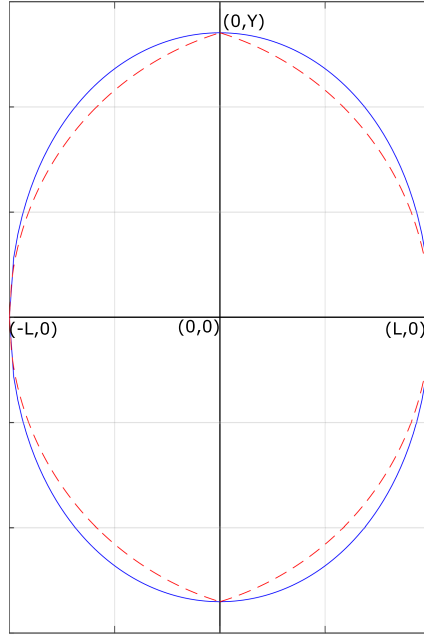


Fig. 3. β and Diversion neighbourhoods for planar and connected embedding.

Similar to the above analysis we can compare neighbourhoods of β -Skeleton in the range $\beta = (0, 1]$ and $SSG(d)$ in the range of $d = [1, 2)$. By analysing distances between points where two neighbourhoods touch each other, we can derive the following theorem for non-planar regions of the $SSG(d)$ and lune-based β -Skeleton.

THEOREM 3.11.

$$\forall d \in [1, 2) \wedge \left(\frac{1}{\beta} - \sqrt{\left(\frac{1}{\beta^2} - 1 \right)} \right) \leq \sqrt{4^{(1-1/d)} - 1},$$

$$SSG(d) \subseteq \text{lune based } \beta\text{-skeleton}.$$

Next, we consider how $SSG(d)$ relates to DT .

THEOREM 3.12. For $d \geq 2$, $SSG(d) \subseteq DT$.

PROOF. By Theorems 3.5 and 3.4, $SSG(\infty) \subseteq SSG(2) \subseteq DT$. Let's assume DT excludes an edge which should be present in $SSG(d)$ for $d \geq 2$. For DT to exclude an edge, endpoints of that edge should not be Voronoi neighbours. An edge between a point pair which are not Voronoi neighbours will not create a planar graph. But, by Lemma 3.6 the excluded edge cannot be in $SSG(d)$ for $d \geq 2$. Thus, our initial assumption is wrong. Hence, by proof of contradiction we establish that for $d \geq 2$, $SSG(d) \subseteq DT$. $\square$

3.3 Algorithms

This section presents the algorithms to compute $SSG(d)$ and how to compute paths based on $SSG(d)$. Based on Theorem 3.12, we can use the complete graph (CG) and the DT graph as starting graphs for extracting $SSG(d)$ for the ranges $2 > d \geq 1$ and $d \geq 2$, respectively. To make the extraction process more efficient, we label the edges spanned by these graph structures with the

minimum value of d necessary for the edge to be excluded from $SSG(d)$. We define the starting graph structure, with edges labelled with these minimum d values, as the d -spectrum.

Definition 3.13 (d -spectrum). For an edge pq , the smallest real number $d \geq 1$ such that $DN(d) \cap V \setminus \{p, q\} \neq \emptyset$ is called the d -value for pq . The set of all edges spanned by V , each labelled by its d -value, is called the d -spectrum of V , denoted as d -spectrum(V) or simply d -spectrum.

For $d \geq 1$, a naive algorithm can be formulated by evaluating all $z \in V \setminus \{p, q\}$ against each point pair $p, q \in V$ and comparing to find the d -value of pq . All edges which have other points on them will be discarded as they will never be present in $SSG(d)$. For the cases where an open lune neighbourhood is empty for an endpoint pair, the edge is labelled with ∞ as its d -value. All remaining edges will be labelled with d -values in the range of $1 \leq d < \infty$. For a V with n points, the number of edges that connect each point pair is in $O(n^2)$. As each edge is compared against the other $(n - 2)$ points, the time complexity of this naive algorithm is $\Theta(n^3)$.

3.3.1 Planar d -Spectrum(V). For $d \geq 2$, DT can be used as a starting point due to Theorem 3.12. As a naive algorithm, for each point pair $p, q \in V$, where there is an edge between those points in DT , all $z \in V \setminus \{p, q\}$ can be evaluated to find the d -value of edge pq . For a V with n number of points, number of edges in DT is $O(n)$. As each edge is compared against the other $(n - 2)$ points, the time complexity of this algorithm is $\Theta(n^2)$.

A more efficient algorithm to obtain the d -Spectrum of edges spanned by $DT(V)$ is shown in Algorithm 1. In this approach, we are sweeping the triangles in $DT(V)$ that are intersecting with the relative neighbourhood of each $p, q \in V : pq \in DT(V)$. The neighbouring triangles of $pq \in DT(V)$ are represented by the set $\Lambda(pq)$. $\Omega(T)$ denote the circumscribed circle of a triangle T . $I(DN(d))$ denote the interior of $DN(d)$, while $B(DN(d))$ denote the boundary of $DN(d)$. We start the process for each $pq \in DT(V)$ with one neighbouring triangle of pq (line 7). And we continue sweeping the triangles intersecting with the line segment connecting the center point of pq and Z_{max} which is the furthest away point in $DN(d)$ from pq . Given the Delaunay triangulation, this algorithm runs in $O(n)$ time as it only examines the points in the relative neighbourhood of each edge.

When presenting the time complexities of proposed algorithms above, we assume that the time taken to solve for the d -value of an edge is $O(1)$, i.e., that we ask for constant precision. Note that all of the above algorithms are readily parallelizable as there is no race condition between separate edge evaluations.

3.3.2 Computing $SSG(V, d)$. Once the d -spectrum of V is calculated for a given d range, extracting the edges of $SSG(d)$ is a straightforward process. As d -Spectrum indicates the d -value of each edge, edges that have d -value greater than given d can be filtered out as the edge set of $SSG(d)$. This process will take $O(m)$ time complexity where m is the number of edges resulting from the method used to calculate the d -Spectrum. For $1 \leq d < 2$, m would be $O(n^2)$, and for $2 \leq d$, m would be $O(n)$.

3.3.3 Improving Running Time of $SPG(V, t)$. Using the d -spectrum, we can exclude some of the edges from $SPG(t)$, without calculating the alternative shortest path, to improve the running time of $SPG(t)$ extraction. By Theorem 3.9, the edges of the d -spectrum that have a d -value less than or equal to a given t value, will not be in the $SPG(t)$. This property can be used to skip edges from the shortest path calculation part of $SPG(t)$ extraction algorithm [15]. It should be noted that even though this method improves the running time of the $SPG(t)$ algorithm, it does not reduce the time complexity of the worst case.

3.3.4 Calculating Paths Using $SSG(d)$. After calculating $SSG(d)$, Dijkstra's algorithm can be used to find the shortest paths between any two endpoints. As $SSG(d)$ contains more alternative

ALGORITHM 1: Planar d -Spectrum(V)

Input: V - Filtered locality set
Output: Every edge $pq \in DT(V)$ marked with d -value

```

1   $DT \leftarrow$  create Delaunay Triangulation of  $V$ ;
2  foreach ( $Edge\ pq : pq \in DT$ ) do
3     $d \leftarrow \infty$ ;
4     $ME \leftarrow pq$ ;
5     $C \leftarrow$  Center of  $pq$ ;
6    foreach ( $T \in \Lambda(pq)$ ) do
7       $MT \leftarrow T$ ;
8       $Zmax \leftarrow Zmax \in DN(d) : \max[l_{CZmax}]$ ;
9      while ( $MT \neq \emptyset$ ) do
10       if ( $Zmax \in \Omega(MT)$ ) then
11         break;
12       end
13        $MV \leftarrow$  Vertex  $Z \in MT : z \notin ME$ ;
14       if ( $MV \in I(DN(d))$ ) then
15          $d \leftarrow d : MV \in B(DN(d))$ ;
16         update  $Zmax$ ;
17       end
18        $ME \leftarrow \exists edge\ E \in MT \setminus ME : ME \cap CZmax \neq \emptyset$ ;
19       if ( $ME = \emptyset$ ) then
20         break;
21       end
22        $MT \leftarrow T1 \in \Lambda(ME) : T1 \neq MT$ ;
23     end
24   end
25   set  $d$  as  $d$ -Value of  $pq$ ;
26 end

```

paths compared to $SPG(t)$, paths taken over $SSG(d)$ will be shorter and more similar to real world paths.

3.4 Applications

As shown in the Experiments section later, $SSG(d)$ skeleton itself can be used to represent and process the travel network between a set of related localities. In this scenario, inferred edges represent the space occupied by an entity when moving between locations. Following are some application scenarios that can benefit from $SSG(d)$.

3.4.1 Nearest Neighbour Queries. The created graph structure can be used to search nearest interesting locations from the current location and to get the path to travel in order to get there. For example, this kind of query can be used to find the nearest exit from an event happening in a park. We propose to use breadth-first search starting from the query location and traverse the graph until a required interesting point is found.

Definition 3.14 (Nearest Neighbour (NN)). A location (L) from an interesting location set (IL) such that the shortest path distance from current location to that location is smaller compared to

the shortest paths to all other locations,

$$NN = \{\exists L \in IL : \forall x \in IL, \text{length}(\text{path}(L)) \leq \text{length}(\text{path}(x))\}.$$

Similarly, we can calculate k-nearest neighbours. Instead of stopping breadth-first search when the first interesting point is found, it can be continued until k interesting points are found. As for the edge weights, we can use weights calculated in Section 3.4.6 according to the usage of edges. This will make sure that the most popular path to the nearest neighbour will be found.

3.4.2 Reverse Nearest Neighbour Queries. *SSG(d)* graphs structure can be used to perform reverse nearest neighbour queries. These queries focus on the inverse relation among a point and a point set. The objective of a reverse nearest neighbour query is to find the set of points which has a given point as their nearest neighbour. This kind of query can be used to evaluate locations to place attractions of an event relative to entrances of the event.

Definition 3.15 (Reverse Nearest Neighbour (RNN)). A set of locations (*RNN*) from an interesting location set (*IL*) such that the current location (*x*) is the nearest neighbour of all locations in *RNN*,

$$RNN = \{\forall L \in IL : x = NN\}.$$

We propose an algorithm to calculate *RNN* of a given location (*x*) utilising breadth-first search. First, breadth-first search is initialized and run from all locations in *IL*. Then, all locations that have *x* as their nearest neighbour are filtered as the *RNN* of *x*.

3.4.3 Group Nearest Neighbour Queries. Another important query type that can be evaluated using *SSG(d)* is group nearest neighbour queries. Given two sets of points *IL1* and *IL2*, the objective of a group nearest neighbour query is to retrieve a point from *IL1* with the smallest sum of distances to all points in *IL2*. For example, this kind of query can be used to find the nearest attraction of an event where a group of users in different locations can get together.

Definition 3.16 (Group Nearest Neighbour (GNN)). A location (*GNN*) from an interesting location set (*IL1*) such that it minimizes the sum of distances to all locations in another location set *IL2*.

$$GNN = \{\exists L \in IL1 : \forall x \in IL2, \min(\sum \text{length}(\text{path}(x)))\}.$$

Again we utilize breadth-first search to propose an algorithm to calculate *GNN*. First, we calculate k-nearest neighbour from all points of *IL2* to all points in *IL1*. Then, for each point in *IL1* we calculate the sum of path distances from all points in *IL2*. Point or points in *IL1* with the minimum sum of path distances is the *GNN*.

3.4.4 Refined Movement Corridors. Once *SSG(d)* is created using posted localities, user trajectories can be used to refine the created travel network. For each consecutive location pair in user trajectories, the shortest path is determined using *SSG(d)*. With each *SSG(d)* edge the number of trajectories passed through that edge is recorded. We represent movement corridors in the travel network based on edges that contain trajectory count higher than a given threshold.

Definition 3.17 (Usage Counter). When *path* is a sequence of edges traversed by a trajectory trace, for all $pq \in E$, Usage Counter of *pq* (denoted *UC(pq)*) is defined as the trajectory count,

$$UC(pq) = |\{path : pq \in path\}|.$$

One of the problems of using *SSG(d)*, as it is for movement network analysis, is that it does not consider the existence of obstacles. As trajectories do not appear on obstacles such as rivers, incorporating trajectory information into *SSG(d)* allows one to filter edges not used by the trajectories. By filtering edges not used by trajectories, we are able to eliminate edges that do not represent user movement information. In summary, refined movement corridors calculated using *SSG(d)* represents the edge subset of *SSG(d)* used by the trajectories for movement.

3.4.5 Inferring Road Network. The above refined movement corridors finding method can be used to infer the road network in an area where we do not have prior knowledge about the road networks. Ideally for this purpose, we need GPS locations published on the road network. The easiest way to obtain such information is to collect LBSN post published while travelling in vehicles. Using these GPS data and associated trajectories, we can infer the refined movement corridors for the given area and visualize the road network.

3.4.6 Most Popular Paths. After calculating usage counters of $SSG(d)$ edges, they can be used to find the most popular path between locations. To find the most popular path, we calculate edge weight to reflect the popularity of the edge and use the shortest path algorithm to calculate the paths. To ensure edges with more usage have lower weights, we divide the length of the edge by usage counter of that edge. For edges with no usage, edge length multiplied by a constant greater than 1 is used as the edge weight. After calculating edge weights in this manner, the Dijkstra's shortest path algorithm is used to find the most popular path between two locations.

Definition 3.18 (eEdge Weight). For all $pq \in E$, the weight of pq is defined as

$$weight(pq) = \begin{cases} l_{pq}/UC(pq), & \text{if } 0 < UC(pq). \\ l_{pq} \times C : 1 < C, & \text{if } UC(pq) = 0. \end{cases}$$

3.4.7 Tour Recommendation. The above most popular path calculation mechanism can be used to calculate the most popular paths between tourist attractions. Given tourist attractions and user trajectories within a city, first we calculate $SSG(d)$ and edge weights based on usage. Then, we apply the most popular path calculation between any two selected attractions to find the path popular among tourists to travel between the selected attractions. The sequence of tourist attractions that appear on the calculated most popular path is reported as a recommended tour along with the path.

3.4.8 Trajectory Clustering. Trajectory clustering refers to the process of grouping similar trajectory traces together. This process involves deriving a representative trajectory for the identified trajectory group. We can use $SSG(d)$ to identify similar trajectory traces and to derive representative trajectory.

First, we align trajectory traces to be clustered along the edges of the created $SSG(d)$. Then for each trajectory trace, edge set that trajectory trace traverses through is recorded. If two trajectories have C ratio of similar edges, they are considered belonging to the same cluster. The subset of similar edges is taken as the representative trajectory of that cluster.

Definition 3.19 (Trajectory Similarity). When $path$ is a sequence of edges traversed by a trajectory trace, two trajectory traces are considered to be similar if they have C percentage of edges in common with each other,

$$similarity(path_1, path_2) = \frac{|path_1 \cap path_2|}{|path_1|}.$$

Both $similarity(path_1, path_2) \geq C$ and $similarity(path_2, path_1) \geq C$ where $0 \leq C \leq 1$ must be true, for $path_1$ and $path_2$ to be in the same cluster.

This method of trajectory clustering is developed without considering the trajectory directions. But this method can be easily changed to accommodate trajectory direction (i.e., cluster only the trajectories travelling in the same direction). For this, we have to consider the endpoint traversing order in addition to considering the edge similarity by endpoints.

3.4.9 Group Movement Detection. The above method proposed to perform trajectory clustering can be further extended to detect users moving together as groups. In order to detect this, we need

to find trajectory traces that have traversed through same edges within the same time in the same direction. To accomplish this, we propose to calculate the time when a user passed through each location in the path resulting from aligning the trajectory on the $SSG(d)$. After that, similar to the trajectory clustering method we detect not only trajectory traces travelled through the same edges, but also whether they have travelled around the same time and in the same direction. Trajectory traces that have travelled through the same edges, around the same time, and in the same direction are filtered as user groups that travelled together. This proposed method to detect group movement is applicable in both sparse geo-located data such as LBSN data and dense data such as data from automated vehicles. But in our experiments we are considering the situation where LBSN data is used to detect group movements.

One problem we have to deal with when using sparse data for group movement detection is the irregularity of group memberships. This happens because not all members of the group are posting their location at every timestep. The proposed method mitigates this problem by calculating the time when each group member passed through each location in the group's path over the graph. Therefore, automatically we assume that a group member has passed through the group path location at the calculated time.

Note that in this method, we consider only users travelled on the same path. This method can be further extended to detect user groups travelled not only on the same path but through nearby paths going in the same direction. This extension can be used to identify flock movement [9].

4 EXPERIMENTS

We perform experiments to see the effectiveness of $SSG(d)$ against $SPG(t)$ on inferring movement networks.

4.1 Datasets

We conducted experiments on two real world LBSN datasets and one synthetic dataset. The first dataset consists of geo-located posts collected from Twitter. A post set collected from 6th March to 23rd April 2012, within a bounding box over the countries Australia and New Zealand is used as the dataset for our analysis. It contains 724, 651 LBSN posts authored by 36, 639 users. For our analysis a LBSN post is defined as a tuple containing four elements: userID to identify authoring user, voluntarily generated textual content, timestamp at which and locality where the post was authored.

We used Yahoo! Flickr Creative Commons 100M (YFCC100M) dataset [22] as the second dataset. It contains metadata such as user information, timestamp, and location where a photo was taken of 100 million photos and videos shared on site Flickr. Only the entries with point geo-locations were used for our experiments. For refined movement corridor and tour recommendation experiments, localities in London, England are filtered out from the YFCC100M dataset.

A synthetic dataset for our experiments was generated using SMARTS simulator [18]. We simulated vehicle movement in Melbourne central business district (CBD) and collected GPS locations of the vehicles every 0.5 seconds. The dataset used for experiment contained 100,000 GPS points. This is essentially a dense GPS dataset. In order to make it sparse, we took a portion of this dataset. Data point filtering is further explained in the road network inferring experiment.

4.2 Implementation

To visualise the inferred neighbourhood skeletons, a visualisation tool was implemented utilizing GeoTools¹ Java libraries. All the skeleton visualisations presented in all figures were generated using this tool. Both $SSG(d)$ and $SPG(t)$ algorithms were implemented using Java 8. The dataset

¹<http://www.geotools.org/>.

was stored in a MongoDB database and a unique locality set relating to a given event was queried from it as V . We used a travel network filtered from OpenStreetMap (OSM)² to get a better understanding of the area being analysed. When filtering a travel network of an area from OSM data we took foot paths, cycle paths, roads, and railroads.

To infer planar $SSG(d)$, firstly, DT was created using the SweepHull [21] algorithm in $O(n \log n)$ time. Then, planar d -Spectrum is calculated using Algorithm 1. Finally, $SSG(d)$ is extracted from d -Spectrum created using DT . We used numerical analysis to calculate the d -value of an edge. More specifically, a Java method was implemented to perform the Secant method³ to approximate the d -value. For extracting planar $SPG(t)$, we used Algorithm 2 presented in [15]⁴. For varying $SSG(d)$ and $SPG(t)$ skeleton extractions, the created DT was reused.

4.3 Event Analysis

An event can be viewed as an extraordinary spatiotemporal phenomenon that motivates LBSN users to generate posts within the spatial and temporal bounds of the event with content identifying the event. Following this perspective, we define an event as a tuple containing three elements: a spatial bound, a temporal bound, and a description of content identifying the event. As the Twitter data we are using has textual content, we use a regular expression as the description of the event.

To discuss the properties of $SSG(d)$ and $SPG(t)$ skeletons, we selected a locality set relating to EasterFest 2012⁵, an annual music festival held in Toowoomba, Australia, from the Twitter dataset. For temporal bound of EasterFest, we took the time period between the 4th and 10th of April 2012. As for the spatial bound of EasterFest, we considered a bounding box (indicated by dashed lines in Figure 1(a)) over the Queens Park, Toowoomba and surrounding main streets where the event was held. We consider all users who have posted with the “#EasterFest” hashtag within spatial and temporal bounds of the event as the user set influenced by the event.

The locality filtering process can be seen as a three-step process. First, the users who have authored content matching the description of the event within the event’s temporal and spatial bounds are filtered. Second, the LBSN trajectories within the event’s temporal bounds generated by the users filtered in the first step are taken as the LBSN posts set influenced by the event. Third, a unique locality set from posts filtered in the second step is taken as the V for variable skeleton generation. Relating to Easterfest, 183 unique points were filtered. Figure 1(a) depicts the filtered locality set along with the travel network of the area analysed. We used the rectangle marked with the dashed lines as the spatial bound of the event. This rectangle encloses the event venue (Queens Park, Toowoomba) and the main streets around it as described above.

4.4 Comparison of Two Graphs

As experiments for filtered locality set (V) relating to EasterFest, we generated varying skeletons with $SSG(d)$ and $SPG(t)$ at $d = t$. After that, generated skeletons are analysed to compare the changing characteristics of the skeletons as parameters vary. We used a locality set relating to an event because all users participating in the event are there for a common reason and exhibit a similar movement pattern. To further understand how graph structures behaved, the number of edges and spanning ratio [2] of the graph structure were plotted with varying configuration parameters (Figure 4).

²<https://www.openstreetmap.org/>.

³https://en.wikipedia.org/wiki/Secant_method.

⁴However, we noticed that $\geq$ in line 5 violates the edge selection criteria proposed as equality allows paths with similar weights to exist between an edge pair. Therefore, we removed equality in line 5 when creating our implementation.

⁵<http://www.easterfest.com.au>.

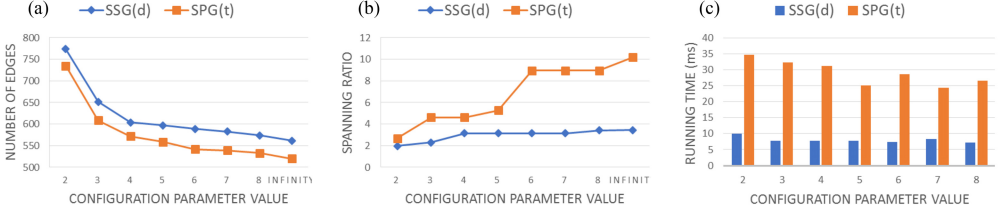


Fig. 4. (a) Number of edges, (b) spanning ratios, and (c) execution times of $SSG(d)$ and $SPG(t)$ when configuration parameters d and t vary.

Figure 1(a) and (b) present the locality set used as the V and the DT skeleton. Figure 1(c) and (e) represent the $SPG(t)$ with t set to 2 and ∞ , respectively. While two skeletons, Figure 1(d) and (f), represent $SSG(d)$ with d set to 2 and ∞ , respectively.

At first glance, we can see that $SPG(t)$ is a subgraph of $SSG(d)$ when $d \leq t$ (Theorem 3.9). Also, in both $SSG(d)$ and $SPG(t)$, edge counts are monotonically reducing (Figure 4(a)). Furthermore, the number of edges in $SPG(t)$ is always smaller than that of $SSG(d)$. This is to be expected as $SPG(t)$ extraction uses a global criteria compared to $SSG(d)$ and infers less edges. Figure 4(b) shows the variation of spanning ratio as configuration parameter varies to demonstrate how the shortest path distances between locality pairs change. The spanning ratio of a graph indicates the maximum ratio between the shortest path distance over the graph and direct distance between any point pair. Therefore, graphs with low spanning ratio are preferred to represent movement networks [2]. Since $SSG(d)$ has a low and stable spanning ratio compared to $SPG(t)$, $SSG(d)$ is suitable for movement analysis. Furthermore, when $SPG(t)$ and $SSG(d)$ are created with the same number of edges, $SPG(t)$ tends to have a higher spanning ratio compared to $SSG(d)$.

Next, we conduct an experiment to see how similar the paths from $SSG(d)$ and $SPG(t)$ are when compared to real world paths taken from OpenStreetMaps. We randomly selected 10 point pairs from the filtered point set and calculated path distances between them using OpenStreetMap paths, $SPG(3)$ and $SSG(3)$. We selected 3 as the configuration parameter for both $SSG(d)$ and $SPG(t)$ as it provides a general movement network between localities. For nine point pairs, paths taken using $SSG(3)$ were different by 20% or less by length compared to real world path taken from OpenStreetMap. However, for paths taken using $SPG(3)$, only six pairs were in the same bracket. Therefore, we can see that paths inferred using $SSG(d)$ are more similar to real world paths compared to paths inferred using $SPG(t)$.

We also calculated nearest neighbour queries using created graph structures. As we used breadth-first search for querying nearest neighbours, the observed results were similar to the results of the shortest path calculations above. Paths calculated to nearest neighbours using $SSG(d)$ were more similar to real world paths compared to paths calculated using $SPG(t)$.

When analysing the spectrum of graph skeletons presented in Figure 1, we can see that as configuration value increases, inferred $SSG(d)$ skeleton emphasises more on tight locality clusters, by thinning longer edges. Relating to the selected event by analysing tweets, it can be seen that filtered localities are densely centered towards the Points of Interest (POIs) such as event location and main streets in the area. This phenomenon makes it easier to visualize POIs clearer, along with the possible movement network between POIs. Furthermore, when comparing inferred skeletons against the road maps available via public sources, it is observed that inferred edges of both $SSG(d)$ and $SPG(t)$ align better with road network as locality density increases in roads. But, as locality density varies, $SSG(d)$ skeleton highlights more alternative paths compared to $SPG(t)$ with the same configuration value.

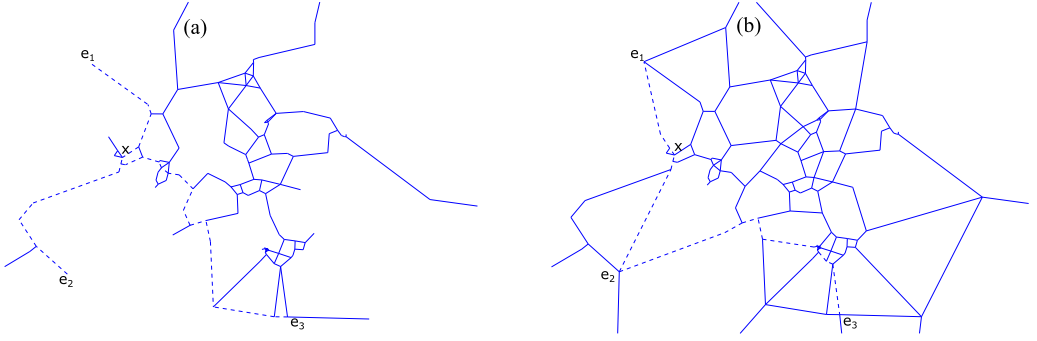


Fig. 5. Dashed lines from e_1 , e_2 , and e_3 indicate their nearest neighbour calculated using (a) $SPG(3)$ and (b) $SSG(3)$.

In order to generate varying visualisations, the skeleton needs to be visualised with varying parameters manually. The times taken to calculate skeletons of $SSG(d)$ and $SPG(t)$ are shown in Figure 4(c). The execution time for $SSG(d)$ calculation is less compared to $SPG(t)$ for all configuration values. To generate $SSG(d)$ skeleton of a given point set, we need to calculate the d -spectrum once and can extract skeleton for desired d from it. Once d -Spectrum is calculated, $SSG(d)$ skeleton can be rendered in $O(m)$ time, where m is the number of edges spanned by V . Therefore, we can generate varying $SSG(d)$ skeletons efficiently. For $SPG(t)$, we have to execute iterative shortest path algorithm every time t changes, making the analysis process inefficient. But, we can reduce this time using calculated d -Spectrum. However, this does not reduce the worst case time complexity of the $SPG(t)$ algorithm.

4.5 Further Interesting Observations From Spatial Queries

4.5.1 Reverse Nearest Neighbour. Reverse nearest neighbour queries aim to find given a query location and a location set, locations of the set where query location is their nearest neighbour. When proposing applications of the $SSG(d)$, we proposed an algorithm to retrieve reverse nearest neighbours of a given query location utilizing breadth-first search. In order to experiment the suitability of $SSG(d)$ for reverse nearest neighbour queries, we have selected an arbitrary location from the venue of the Easterfest 2012 as the query location and exits of the venue as the location set.

We executed the reverse nearest neighbour query on both $SPG(3)$ and $SSG(3)$. Results are shown in Figure 5. For both exits e_1 and e_2 , query location x became the nearest neighbour in both $SPG(3)$ and $SSG(3)$. For e_3 , nearest neighbour results were different for two graphs. With $SPG(3)$, x became the nearest neighbour for e_3 . With $SSG(3)$, e_2 became the nearest neighbour for e_3 . Therefore, with $SPG(3)$, reverse nearest neighbours of x are e_1 , e_2 , e_3 , while with $SSG(3)$ reverse nearest neighbours of x are e_1 , e_2 . In Euclidean space, the nearest neighbour of e_3 is e_2 . Therefore, we can see that $SSG(d)$ is more suitable for reverse nearest neighbour queries compared to $SPG(t)$ with the same configuration parameter. Also note that paths selected from exits to query location are shorter in $SSG(3)$ compared to $SPG(3)$.

4.5.2 Group Nearest Neighbour. Group nearest neighbour queries are evaluated between two sets of locations. The aim of this query type is to find a location or locations from one set such that cumulative travel distance from all locations in the other set is minimized. Our proposed algorithm to calculate group nearest neighbour is to calculate k -nearest neighbour from all locations in one set to find all locations in the other set and evaluate which location in the second location set results in the least cumulative travel distance from all locations in the first set. To evaluate group nearest neighbour queries, we selected the same location set filtered for Easterfest 2012.

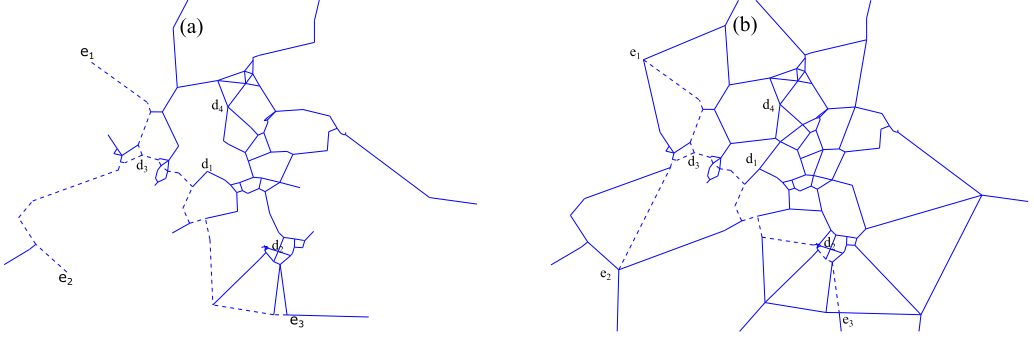


Fig. 6. Dashed lines from e_1, e_2 , and e_3 indicate their group nearest neighbour calculated using (a) $SPG(3)$ and (b) $SSG(3)$, between destinations d_1, d_2, d_3 , and d_4 .

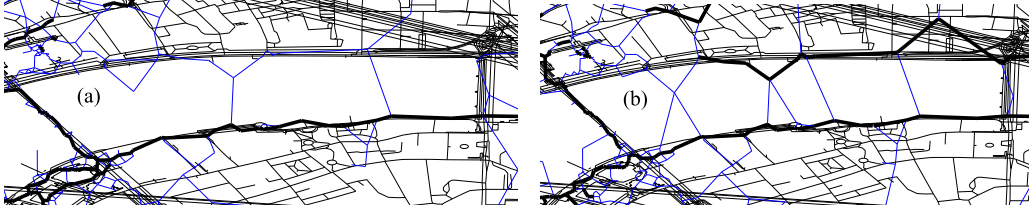


Fig. 7. Thick lines indicate refined movement corridors extracted using (a) $SPG(3)$ and (b) $SSG(3)$.

We experimented performing group nearest neighbour queries on both $SPG(3)$ and $SSG(3)$. Results are shown in Figure 6. We calculated group nearest neighbour from three exits (e_1, e_2 , and e_3) to four arbitrary locations within the event venue (d_1, d_2, d_3 , and d_4). Using both graphs $SPG(3)$ and $SSG(3)$, destination d_3 was calculated as the group nearest neighbour. Dashed lines in Figure 6 indicate the path calculated from an exit to the group nearest neighbour. Even though both graphs resulted in same group nearest neighbour, note that paths calculated using $SSG(3)$ are shorter compared to paths calculated using $SPG(3)$ due to the availability of more alternative paths. Hence, we can see that $SSG(d)$ is more suitable for group nearest neighbour calculations compared to $SPG(t)$ with the same configuration parameter.

4.5.3 Refined Movement Corridors. Refined movement corridors refer to edges of the graph that are used for movement. These edges are selected by aligning user trajectories along the graph edges using shortest path calculation. We analysed refined movement corridors relating to the trajectories filtered from the YFCC100M dataset, around Thames river in London. To represent the travel networks, $SPG(3)$ and $SSG(3)$ were used. This dataset was selected because it had a lot of tourist movement compared to the Twitter dataset. After that, trajectories are aligned along both graph skeletons, and all the edges with usage counter more than 5 are filtered as refined movement corridors. Intuitively, if an edge is used by trajectories five times, that edge is selected as a refined movement corridor. Visualisations created using filtered edges are shown in Figure 7. In both, refined movement corridors represented by thick edges appear along the banks of the river and on bridges where movements happen. Edges created across the river are filtered out as there is no movement happening. When all trajectories are aligned against the refined movement corridors calculated using $SSG(3)$ and $SPG(3)$, 14% more of trajectories appear on refined movement corridors calculated using $SSG(3)$ compared to $SPG(3)$. Hence, refined movement corridors created using $SSG(d)$ are more similar to real world paths compared to refined movement corridors

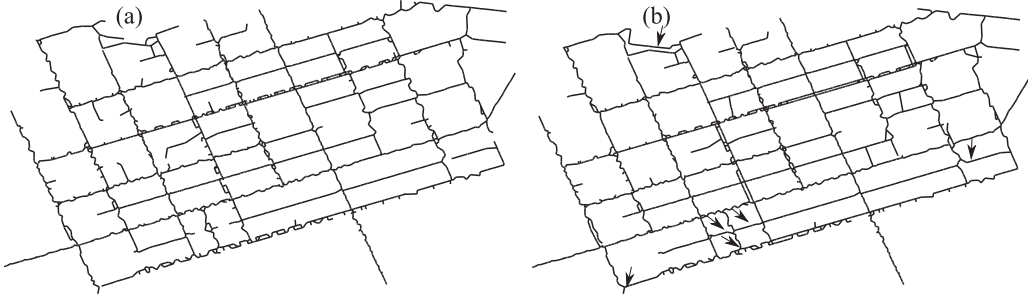


Fig. 8. Road network (refined movement corridors) extracted using synthetic data on (a) *SPG*(3) and (b) *SSG*(3). Arrows in (b) indicate additional movement corridors inferred using *SSG*(3).

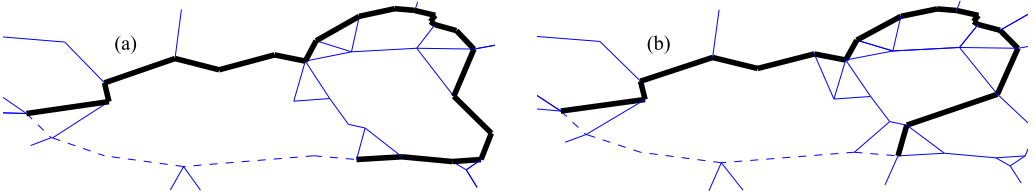


Fig. 9. Paths calculated between two arbitrary locations using (a) *SPG*(3) and (b) *SSG*(3). The dashed line indicates the shortest path and the thick line indicates the most popular path. Note that the most popular path calculated with *SSG*(3) is shorter than that of *SPG*(3).

calculated using *SPG*(t). As this method indicates the most used part of the travel network, it can be used to generate the preferred part of a travel network used for calculating preferred paths [19].

4.5.4 Inferring Road Network. Refined movement corridor extraction can be used to infer the road network of an area. GPS locations posted on the road network are useful for this purpose. Using our synthetic dataset, we inferred the road network of the Melbourne CBD. In order to make this data sparse similar to LBSN data, we filtered out some of the points. The filtering process was to order all GPS points based on the timestamp and take every n^{th} point from the whole dataset. However, we used all the trajectories for the refined movement corridor calculation.

In Figure 8, we have shown road networks inferred using (a) *SPG*(3) and (b) *SSG*(3). As shown in the figure, road network inferred using *SPG*(3) breaks around some junctions, but *SSG*(3) recreates the junctions correctly. Overall, 12% more road segments inferred using *SSG*(3) aligns with real road network compared to *SPG*(3). It should be noted that as n grows, the dataset used to infer road network becomes more sparse, therefore results generated using this method degrade. Results heavily degrade when n reaches around 120.

4.5.5 Most Popular Paths. After calculating refined movement corridors over an area, edge weights are set to exhibit the usage of that edge as shown in Section 3.4.3. The most popular paths between two locations can be found by calculating the shortest path using the above calculated edge weights. In order to experiment the effectiveness of the most popular path finding mechanism, we applied it to find a path around a junction in Melbourne CBD. Data for the experiment is taken from the Twitter dataset. As the Twitter dataset was restricted to the region Australia, it has a lot of movement information of Twitter users relating to their daily travels. One example of the most popular path is shown in Figure 9. *SPG*(3) and *SSG*(3) were used to represent travel networks, respectively. Calculated shortest paths are shown in broken lines and the most popular

Table 1. Comparison of Tour Recommendations Calculated Using *SPG*(3) and *SSG*(3)

Graph	Recall	Precision	F ₁ -score
<i>SPG</i> (3)	0.699± 0.050	0.681± 0.060	0.676± 0.050
<i>SSG</i> (3)	0.777± 0.050	0.783± 0.060	0.774± 0.059

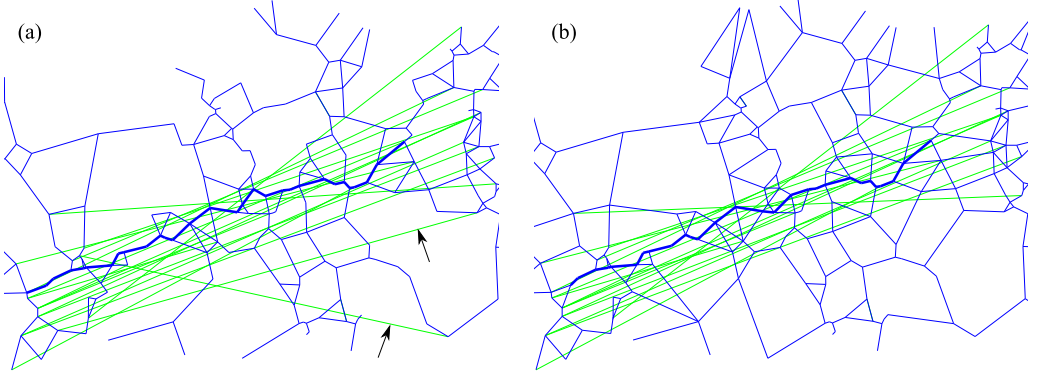


Fig. 10. Thick lines indicate representative trajectory extracted for trajectory set shown as green (light-coloured in black and white print) lines using (a) *SPG*(3) and (b) *SSG*(3). Additional trajectory traces that belong to the *SPG*(3) cluster are marked with arrow heads.

paths are shown in thick edges. As shown in Figure 9, the most popular path inferred using *SSG*(3) is shorter compared to that of *SPG*(3). In fact, when most popular paths are calculated for all trajectories, most popular paths calculated using *SSG*(3) were 18% shorter than most popular paths calculated using *SPG*(3). In this particular case when selected most popular paths are compared with travel network taken from OpenStreetMaps, it is seen that there are roads that align with the most popular path, whereas no actual road aligns with shortest paths.

4.5.6 Tour Recommendation. Using the YFCC100M dataset, we experimented the effectiveness of tour recommendation in the city of London. We followed the experimental setup followed in [13]. Geo-located photos posted over a 2 month time period were used for experiments. All tourist travel sequences set apart by 1 hour time intervals are taken as the query dataset. For each tourist attraction sequence travelled by users, we calculated the most popular path between first and last tourist attractions they've been to and selected tourist attractions that coincide with the most popular path as recommended tour itinerary. For each real world travel sequence (P_v), a recommended tour itinerary (P_r) was calculated using both *SPG*(3) and *SSG*(3). For tours recommended by both graphs tour recall ($T_R = |P_r \cap P_v|/|P_v|$), tour precision ($T_P = |P_r \cap P_v|/|P_r|$) and tour F₁-score ($T_{F1} = (2 \times P_r \times P_v)/P_v \times P_r$) [13] were calculated and presented in Table 1. As shown for all three criteria, *SSG*(3) evaluates higher values than *SPG*(3). Thus, we can say that the inferences using *SSG*(d) are closer to real world user travels than *SPG*(t).

4.5.7 Trajectory Clustering. The proposed *SSG*(d) can be used for trajectory clustering by aligning trajectories on the created graph and selecting trajectory traces that share the same edge sets from the graph. To evaluate *SPG*(t) and *SSG*(d) for trajectory clustering, we used trajectory data collected in Melbourne CBD from Twitter dataset. A trajectory cluster detected in the same area using both *SPG*(3) and *SSG*(3) is shown in Figure 10. We set similarity threshold value C to 0.5. Green (light colour in black and white print) lines in the figures indicate trajectory traces included

in the cluster, while thick lines indicate the representative trajectory calculated using graphs consisting of the shared edge set among trajectories. The trajectory cluster calculated using $SPG(3)$ contains 15 trajectory traces while the cluster calculated using $SSG(3)$ only contains 13 traces. Two additional trajectory traces that belong to the cluster found using $SPG(3)$ are marked by arrows. The reason for the $SSG(3)$ cluster leaving two traces out is due to the availability of alternative paths trajectory traces that deviate from other trajectory traces in the cluster can find paths other than the paths used by the trajectories in the cluster. Since $SPG(3)$ is more restrictive than $SSG(3)$, trajectory traces that deviate from most of the trajectory traces in the cluster have to travel in the same paths used by the clustered trajectory traces. Following the quality measures developed in [12] to measure the quality of trajectory clusters, we developed a quality measure. Once trajectory traces are aligned along the edges of the graph, we take the lengths of edges in the trajectory that are outside the representative trajectory of the cluster and divide the cumulative outside length of all the edges by the number of trajectories in the cluster. By definition of this measure, the lower the calculated value for this measure, the better the cluster. With this measurement, the trajectory cluster shown in Figure 10, clustered using $SSG(3)$, was 13% better than the cluster identified using $SPG(3)$. Therefore, we can say $SSG(d)$ is more suitable to identify similar trajectory clusters compared to $SPG(t)$.

4.5.8 Group Movement Detection. This application refers to the use of $SSG(d)$ to detect user groups moving together. This is an extension of the trajectory clustering presented earlier. Instead of grouping trajectories based only on their spatial proximity, in the group movement detection we consider their direction and travelled time. Using this information we find trajectories travelled in the same path, in the same direction, around the same time.

In both datasets we used, there were not any group movements that exhibit the difference between group movement detection using $SPG(t)$ and $SSG(d)$, due to the small size of data we have from real LBSN leading to very few group formations. Therefore, we created a synthetic dataset using a geo-located Twitter post dataset to exhibit this difference. In this synthetic dataset, locations of the geo-located posts are not altered. However, timestamps and user ids are altered to make synthetic trajectories.

With this synthetic dataset using both graphs $SPG(3)$ and $SSG(3)$, we detected a group movement occurred in Melbourne CBD. In the detected group movement with $SPG(3)$ there are trajectories of five users aligning with the same path of the graphs travelling in the same direction with a time difference of 10 minutes. However, in the group movement detected using $SSG(3)$ there are only four users. Detected group movement paths are shown in Figure 11. Thick lines in the graphs indicate edges of the path shared by the detected group. Lines in green colour (light-colour lines in black and white print) indicate trajectories of the users that travelled together. In the $SPG(3)$ case, there is a trajectory marked with an arrow head that is not travelling with other trajectories. This trajectory is clustered with other trajectories that exhibit group movement because of the restrictiveness on the $SPG(t)$. Therefore, group movement detection using $SPG(t)$ can suffer from the same weaknesses that would hinder its performance in trajectory clustering. Due to the greater restrictiveness of $SPG(t)$, trajectories that are not supposed to travel in the same path may end up travelling in shared paths. This is avoided in $SSG(d)$ due to the availability of more alternative paths. Therefore, $SSG(d)$ is more suitable to detect group movements using the proposed method.

4.6 Configuration Value

We selected 3 as the configuration value to compare resulting graphs generated using $SSG(d)$ and $SPG(t)$. For configuration value 1, both $SSG(d)$ and $SPG(t)$ are the same and do not cull edges to

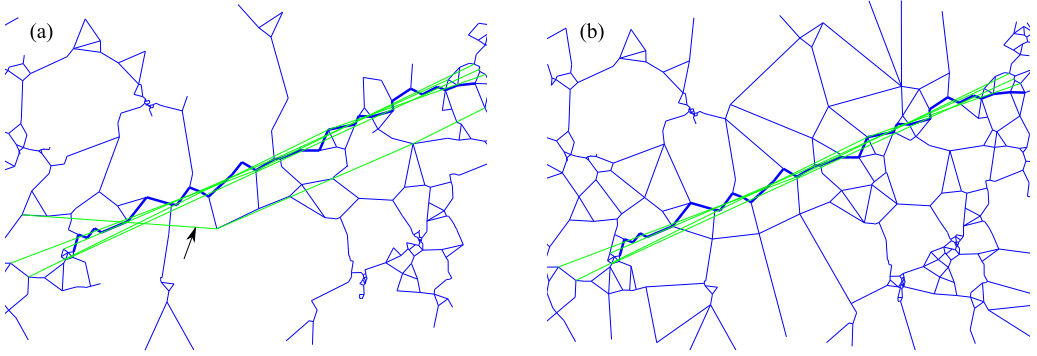


Fig. 11. Thick lines indicate the path of group movement detected (from left to right) using (a) $SPG(3)$ and (b) $SSG(3)$.

cater for point density. And for configuration parameter value 2, $SSG(d)$ becomes GG , which is a well-known static graph. Configuration parameter value 3 gives interesting results, so we have used it. Configuration values greater than 3 give similar results as the results generated with the configuration parameter set to 3.

5 FUTURE DIRECTIONS

By analysing experimental results, we can observe that $SSG(d)$ is both effective and efficient for representing and processing the possible movement network. Therefore, the proposed analysis method can be used in situations such as analysing movement patterns of animals where similar locality distributions are observed. In our experiment, only the spatial and temporal aspects of the LBSN data are considered for analysis without geographic information of the area. Furthermore, the parameter selection for variable skeleton generation is considered as a manual process. Following future development can be highlighted based on our experiment.

- Autonomous detection of a suitable value for parameter d .
- Investigation on information that can be visualized using other aspects such as the sentimental aspect of LBSN data.
- Development of colouring overlays based on the created skeleton.

Analysis of trajectories resulting from moving objects based on $SSG(d)$ can be listed as a promising future direction. Furthermore, $SSG(d)$ can be used for movement planning by filtering out edges longer than a given threshold [5]. Also, $SSG(d)$ skeletons can be useful in detecting herding behaviour in emergency evacuation situations [1]. With the introduction of $SSG(d)$ as a super graph of $SPG(t)$ when $d \leq t$, it is worth investigating how use cases of $SPG(t)$ can be further improved. Since $SSG(d)$ emphasizes on tight locality clusters as d increases, we maybe able to break the connectivity property of the graph and identify clusters.

Since we are analysing geographic data, we limited the $SSG(d)$ definition to the two-dimensional case with Euclidean distance. As $DN(d)$ definition is distance based, the concept of $SSG(d)$ can be extended to higher dimensions. As planarity of the extracted skeleton is observed to be a desired property, we only presented results with a configurable value range of $d \geq 2$. But, there are use cases that can benefit from value range $d < 2$ [17]. Therefore, research on how the non-planar region of $SSG(d)$ behaves is another interesting direction.

With the trajectory clustering proposal, it is important to compare it against other trajectory clustering mechanisms such as TRACCLUS [12]. As further development, we can combine features

from both approaches such as calculating shared paths using $SSG(d)$ and distance function from TRACCLUS to develop new trajectory clustering mechanisms to cluster trajectories in nearby shared paths. Group movement detection using $SSG(d)$ can be extended to conduct flock analysis using LBSN data. We can further process detected group movement to track the movements and predict future movements of groups. In this future direction, we have to handle complex movement scenarios such as merging and splitting of groups.

6 CONCLUSION

We analysed the suitability of variable connected neighbourhood skeletons to visualize the possible movement network of LBSN users influenced by events. We presented the Stepping Stone Graph ($SSG(d)$) of a finite planar set, which is a connected graph that varies depending on a single parameter d . We analysed how $SSG(d)$ relates to some well-known graph structures, and we presented algorithms to calculate $SSG(d)$. In addition, we presented how $SSG(d)$ can be used to improve the running time of the Shortest Path Graph ($SPG(t)$). We have empirically shown that $SSG(d)$ is both efficient and effective to visualize the possible movement network using LBSN data due to its distance-based local evaluation criteria. Furthermore, we have shown the usefulness of $SSG(d)$ in a few applications.

REFERENCES

- [1] David Amores, Maria Vasardani, and Egemen Tanin. 2018. Early detection of herding behaviour during emergency evacuations. In *10th International Conference on Geographic Information Science (GIScience'18 (LIPICs)*, Stephan Winter, Amy Griffin, and Monika Sester (Eds.), Vol. 114. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 1:1–1:15.
- [2] Prosenjit Bose, Luc Devroye, William S. Evans, and David G. Kirkpatrick. 2006. On the spanning ratio of Gabriel graphs and beta-skeletons. *SIAM Journal on Discrete Mathematics* 20, 2 (2006), 412–427.
- [3] Jean Cardinal, Sébastien Collette, and Stefan Langerman. 2009. Empty region graphs. *Computational Geometry* 42, 3 (2009), 183–195.
- [4] Junghoon Chae, Dennis Thom, Yun Jang, SungYe Kim, Thomas Ertl, and David S. Ebert. 2014. Public behavior response analysis in disaster events utilizing visual analytics of microblog data. *Computers & Graphics* 38 (2014), 51–60.
- [5] Jorge Cortés, Sonia Martínez, and Francesco Bullo. 2006. Robust rendezvous for mobile autonomous agents via proximity graphs in arbitrary dimensions. *IEEE Transactions on Automatic Control* 51, 8 (2006), 1289–1298.
- [6] Mark de Berg, Wouter Meulemans, and Bettina Speckmann. 2011. Delineating imprecise regions via shortest-path graphs. In *Proceedings of the 19th ACM SIGSPATIAL International Symposium on Advances in Geographic Information Systems (ACM-GIS'11)*, Isabel F. Cruz, Divyakant Agrawal, Christian S. Jensen, Eyal Ofek, and Egemen Tanin (Eds.). ACM, 271–280.
- [7] Matt Duckham, Marc J. van Kreveld, Ross Purves, Bettina Speckmann, Yaguang Tao, Kevin Verbeek, and Jo Wood. 2016. Modeling checkpoint-based movement with the Earth Mover's Distance. See Miller et al. [16], 225–239.
- [8] K. Ruben Gabriel and Robert R. Sokal. 1969. A new statistical approach to geographic variation analysis. *Systematic Biology* 18, 3 (1969), 259–278.
- [9] Hoyoung Jeung, Man Lung Yiu, and Christian S. Jensen. 2011. Trajectory pattern mining. In *Computing with Spatial Trajectories*, Yu Zheng and Xiaofang Zhou (Eds.). Springer, 143–177.
- [10] Sameera Kannangara, Egemen Tanin, Aaron Harwood, and Shanika Karunasekera. 2018. Stepping stone graph for public movement analysis. In *Proceedings of the 26th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (SIGSPATIAL'18)*, Farnoush Banaei-Kashani, Erik G. Hoel, Ralf Hartmut Güting, Roberto Tamassia, and Li Xiong (Eds.). ACM, 149–158.
- [11] David G. Kirkpatrick and John D. Radke. 1985. A framework for computational morphology. In *Machine Intelligence and Pattern Recognition*. Vol. 2. Elsevier, 217–248.
- [12] Jae-Gil Lee, Jiawei Han, and Kyu-Young Whang. 2007. Trajectory clustering: A partition-and-group framework. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, Chee Yong Chan, Beng Chin Ooi, and Aoying Zhou (Eds.). ACM, 593–604.
- [13] Kwan Hui Lim, Jeffrey Chan, Christopher Leckie, and Shanika Karunasekera. 2015. Personalized tour recommendation based on user interests and points of interest visit durations. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI'15)*, Qiang Yang and Michael Wooldridge (Eds.). AAAI Press, 1778–1784.
- [14] Alan M. MacEachren, Anuj R. Jaiswal, Anthony C. Robinson, Scott Pezanowski, Alexander Savelyev, Prasenjit Mitra, Xiao Zhang, and Justine I. Blanford. 2011. SensePlace2: GeoTwitter analytics support for situational awareness. In

- Proceedings of the 2011 IEEE Conference on Visual Analytics Science and Technology (VAST '11)*. IEEE Computer Society, 181–190.
- [15] Wouter Meulemans, Nathalie Henry Riche, Bettina Speckmann, Basak Alper, and Tim Dwyer. 2013. KelpFusion: A hybrid set visualization technique. *IEEE Transactions on Visualization and Computer Graphics* 19, 11 (2013), 1846–1858.
 - [16] Jennifer A. Miller, David O’Sullivan, and Nancy Wiegand (Eds.). 2016. *Proceedings of the 9th International Conference on Geographic Information Science (GIScience’16)*. Lecture Notes in Computer Science, Vol. 9927. Springer.
 - [17] Joon C. Park, Hayong Shin, and Byoung Kyu Choi. 2006. Elliptic Gabriel graph for finding neighbors in a point set and its application to normal vector estimation. *Computer-Aided Design* 38, 6 (2006), 619–626.
 - [18] Kotagiri Ramamohanarao, Hairuo Xie, Lars Kulik, Shanika Karunasekera, Egemen Tanin, Rui Zhang, and Eman Bin Khunayn. 2017. SMARTS: Scalable microscopic adaptive road traffic simulator. *ACM TIST* 8, 2 (2017), 26:1–26:22.
 - [19] Dimitris Sacharidis, Panagiotis Bouros, and Theodoros Chondrogiannis. 2017. Finding the most preferred path. In *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS’17)*, Erik G. Hoel, Shawn D. Newsam, Siva Ravada, Roberto Tamassia, and Goce Trajcevski (Eds.). ACM, 5:1–5:10.
 - [20] Dipto Sarkar, Renée Sieber, and Raja Sengupta. 2016. GIScience considerations in spatial social networks. See Miller et al. [16], 85–98.
 - [21] David Sinclair. 2016. S-hull: A fast radial sweep-hull routine for Delaunay triangulation. *CoRR* abs/1604.01428.
 - [22] Bart Thomee, David A. Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. 2016. YFCC100M: The new data in multimedia research. *Communications of the ACM* 59, 2 (2016), 64–73.
 - [23] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern Recognition* 12, 4 (1980), 261–268.

Received December 2018; revised April 2019; accepted April 2019